Desafío Técnico

Fullstack Developer Senior

Java Spring Boot • Node.js • React • PostgreSQL

Proyecto: Seller Center

Posición	Fullstack Developer Senior
Stack requerido	Java 17+ (Spring Boot), Node.js, React + TypeScript
Base de datos	PostgreSQL
Cloud	Google Cloud Platform (GCP) o similar
Tiempo estimado	3 días hábiles
Entrega	Repositorio público en GitHub con README detallado

1. Objetivo del desafío

El desafío busca evaluar las habilidades técnicas del candidato en el diseño e implementación de una solución fullstack, cubriendo backend en Java y Node.js, frontend en React y deploy en la nube.

El foco está en la calidad del código, el criterio de diseño y la capacidad de tomar decisiones técnicas de forma autónoma. No se busca exhaustividad de features sino solidez en lo entregado.

2. Guía de distribución de tiempo (orientativa)

Dia 1	**Backend Java (MS-A)**
	Configurar proyecto Spring Boot con conexión a PostgreSQL.
	Implementar los 3 endpoints con validaciones.
	Documentar la API con Swagger.

Dia 2	**BFF Node.js + Frontend React**
	Implementar el BFF en Node.js como proxy hacia el MS-A.
	Desarrollar las 3 pantallas en React + TypeScript consumiendo el BFF.

Dia 3	Deploy, tests y README
	Deploy de los 3 componentes en GCP u otra plataforma cloud. Agregar al menos un test unitario en Java (JUnit) y uno en Node.js (Jest).
	Completar el README con instrucciones, links y decisiones tecnicas.

---

3. Arquitectura general

La solución se compone de tres partes independientes:

- Microservicio A (Java Spring Boot): core de negocio con persistencia en PostgreSQL.
- Microservicio B (Node.js + Express): BFF liviano que actúa como proxy hacia el MS-A.
- Frontend (React + TypeScript): interfaz que consume únicamente _el Microservicio B.

Si el candidato prefiere una arquitectura diferente, puede hacerlo justificándolo en el README. Se valorara el razonamiento técnico.

---

4. Microservicio A — Java Spring Boot

4.1 Tecnologias

- Java 17+ con Spring Boot 3.x
- Spring Data JPA + PostgreSQL
- Swagger / OpenAPI 3
- JUnit 5 + Mockito (al menos un test unitario en la capa de servicio)

4.2 Endpoints requeridos

Metodo	Endpoint	Descripcion	Servicio
POST	/api/clientes	Crear un nuevo cliente	MS-A (Java)
GET	/api/clientes/kpi	Promedio y desvío estandar de edades	MS-A (Java)
GET	/api/clientes	Lista completa + fecha estimada de fallecimiento	MS-A (Java)

4.3 Detalle de cada endpoint

POST /api/clientes

- Body: nombre (string), apellido (string), edad (integer), fechaNacimiento (ISO 8601).
- Validar todos los campos con @Valid / Bean Validation.
- Persistir en PostgreSQL. Retornar 201 Created con el recurso creado.

GET /api/clientes/kpi

- Retorna: promedio Edad y desviación. Estándar calculados sobre todos los clientes registrados.

GET /api/clientes

- Retorna la lista completa de clientes con todos sus campos.
- Incluye fecha Probable Fallecimiento calculada asumiendo una expectativa de vida de 80 anos.

4.4 Calidad mínima esperada

- Código organizado en capas: controller / service / repository.
- Manejo de errores con respuestas descriptivas (no stacktraces).
- Swagger UI accesible en /swagger-ui/index.html.
- Al menos un test unitario con JUnit 5 + Mockito sobre la capa de servicio.

---

5. Microservicio B — Node.js BFF

5.1 Tecnologías

- Node.js 18+ con Express
- JavaScript o TypeScript (se prefiere TypeScript)
- Axios o node-fetch para comunicarse con el MS-A
- Jest (al menos un test unitario)

5.2 Responsabilidad

Este servicio es una capa liviana que:

- Expone los mismos 3 endpoints al frontend.
- Redirige las llamadas al MS-A y retorna las respuestas.
- Centraliza el manejo de errores hacia el frontend.

5.3 Endpoints

Metodo	Endpoint	Descripción	Servicio
POST	/bff/clientes	Proxy hacia POST /api/clientes	MS-B (Node.js)
GET	/bff/clientes/kpi	Proxy hacia GET /api/clientes/kpi	MS-B (Node.js)
GET	/bff/clientes	Proxy hacia GET /api/clientes	MS-B (Node.js)

---

6. Frontend — React + TypeScript

6.1 Tecnologias

- React 18+ con TypeScript
- Vite o Create React App
- Fetch o Axios para consumir el BFF

6.2 Pantallas requeridas

Pantalla 1 — Crear Cliente

- Formulario con campos: Nombre, Apellido, Edad, Fecha de Nacimiento.
- Validaciones básicas en cliente (campos obligatorios).
- Botón que hace POST al BFF. Feedback visual de carga, éxito o error.

Pantalla 2 — Estadísticas (KPI)

- Muestra el promedio de edad y la desviación estándar retornados por el BFF.

Pantalla 3 — Lista de Clientes

- Tabla con todos los clientes: Nombre, Apellido, Edad, Fecha de Nacimiento, Fecha Probable de Fallecimiento.

6.3 Consideraciones

- No se evalúa diseño visual elaborado, si funcionalidad y manejo de estados de carga/error.
- La app debe ser usable en escritorio.

---

7. Despliegue

- Los tres componentes deben estar deployados y accesibles publicamente.
- Plataforma preferida: GCP. También se acepta Railway, Render, AWS, Azure u otra.
- El README debe incluir links directos a cada componente y a Swagger.

Docker y docker-compose son bienvenidos pero opcionales dentro de este plazo.

---

8. Puntos extra (opcionales)

- Docker + docker-compose para levantar toda la solución localmente.
- Variables de entorno con .env (sin valores hardcodeados).
- CI/CD básico con GitHub Actions.
- Mención del uso de herramientas de IA durante el desarrollo en el README.

---

9. Entrega

9.1 Repositorio GitHub

- Repositorio público con el código de los tres componentes.
- README con instrucciones para correr el proyecto localmente.
- Links a los servicios deployados y a Swagger.
- Breve descripción de las decisiones técnicas tomadas.

9.2 Checklist

1. Repositorio GitHub público con código completo.

2. README con pasos para correr el proyecto localmente.
3. Los 3 endpoints del MS-A funcionando y documentados en Swagger.
4. BFF Node.js operativo como proxy del MS-A.
5. Frontend React con las 3 pantallas funcionales.
6. Al menos un test en Java (JUnit) y uno en Node.js (Jest).
7. Deploy publico accesible con links en el README.

---

10. Criterios de evaluación

Area	Criterio	Peso
Backend Java (MS-A)	Correctitud de endpoints, validaciones, estructura del código	**35%**
Backend Node.js (BFF)	Funcionamiento del proxy, manejo de errores, TypeScript	**20%**
Frontend React	Funcionalidad de las 3 pantallas, manejo de carga/error, TypeScript	**25%**
Tests	Al menos 1 test en Java y 1 en Node.js	**10%**
Deploy + README	Servicios accesibles, documentación clara, decisiones justificadas	**10%**

---

11. Notas finales

No existe una única solución correcta. Se valora el criterio técnico, la claridad del código y la capacidad de justificar las decisiones tomadas más que la cantidad de features implementados.

Ante cualquier ambiguedad, el candidato puede dejar en el README los supuestos que asumió. Esto forma parte de la evaluación.

Mucho exito!